

PROYECTO ITV

Laura Santamaría Parra 1DAW

ÍNDICE

1. INTRODUCCIÓN

- 1.1 Propósito del Proyecto
- 1.2 Tecnologías Utilizadas
- 1.3 Alcance del Proyecto

2. ANÁLISIS DE REQUISITOS

- 2.1 Requisitos Funcionales (RF)
 - 2.1.1 Gestión de Citas
 - 2.1.2 Búsqueda y Validación
 - 2.1.3 Importación y Exportación de Datos
 - 2.1.4 Sistema de Backups (Copias de Seguridad)
 - 2.1.5 Interfaz y Navegación
 - 2.1.6 Restricciones de Seguridad
 - 2.1.7 Persistencia de Datos
 - 2.1.8 Configuraciones del Sistema
- 2.2 Requisitos No Funcionales (RNF)
 - 2.2.1 Tecnologías y Entorno
 - 2.2.2 Buenas Prácticas y Arquitectura
 - 2.2.3 Restricciones Técnicas
- 2.3 Requisitos de Información (RI)
 - 2.3.1 Modelo de Dominio
 - 2.3.2 Configuración del Sistema

3. DOCUMENTACIÓN UML

- 3.1 Diagrama de Casos de Uso
- 3.2 Diagrama de Arquitectura
- 3.3 Diagrama de Secuencias
 - 3.3.1 Diagrama Agregar
 - 3.3.2 Diagrama Actualizar
 - 3.3.3 Diagrama Borrar
 - 3.3.4 Diagrama GetAll

4. DISEÑO DE LA BASE DE DATOS

- 4.1 Elección de SQLite
- 4.2 Estructura de la Tabla Cita
- 4.3 Restricciones y Validaciones
- 4.4 Clave Alternativa (Matrícula, FechaInspeccion)
- 4.5 Consideraciones Futuras

5. TESTING

- 5.1 Metodología de Testing
- 5.2 Complejidad Ciclomática
- 5.3 Casos de Prueba (Función Agregar)
- 5.4 Cobertura de Código

6. ANÁLISIS ECONÓMICO

7. MEJORAS A FUTURO

- **7.1 Optimizaciones Arquitectónicas**
- **7.2 Testing y Código**
- **7.3 Base de Datos**
- **7.4 Interfaz de Usuario**
- **7.5 Datos y Validación**

8.LINKS

1. INTRODUCCIÓN

1.1 Propósito del Proyecto

Proyecto de desarrollo en el que se integran tecnologías de arquitectura limpia para realizar una aplicación de escritorio con el propósito de gestionar las citas de inspecciones técnicas de vehículos (ITV).

1.2 Tecnologías Utilizadas

- **Lenguaje de programación:** C# 14
- **Plataforma de ejecución:** .NET 10
- **Entorno gráfico de escritorio:** WPF (Windows Presentation Foundation) y XAML

1.3 Alcance del Proyecto

Desarrollo completo de una aplicación de gestión de citas con soporte multimotores de persistencia, sistema de copias de seguridad, exportación de informes en múltiples formatos y validación estricta de reglas de negocio en base a criterios de calidad y mantenibilidad.

2. ANÁLISIS DE REQUISITOS

Acorde a los requisitos dados, estos se han clasificado de la siguiente manera dentro del análisis del sistema:

2.1 Requisitos Funcionales (RF)

2.1.1 Gestión de Citas

- **RF1:** Añadir una nueva cita en el sistema.
- **RF2:** Borrar una cita existente.
 - **RF2.1:** Borrar una cita existente mediante borrado lógico.
 - **RF2.2:** Borrar una cita existente mediante borrado físico.
- **RF3:** Actualizar los datos de una cita existente.
- **RF4:** Visualizar los detalles de una cita específica.
- **RF5:** Visualizar el listado de citas de forma paginada.

2.1.2 Búsqueda y Validación

- **RF6:** Buscar citas aplicando filtros basados en cualquier dato introducido.
- **RF7:** Validar los campos del formulario y mostrar los mensajes de error o éxito correspondientes.

2.1.3 Importación y Exportación de Datos

- **RF8:** Exportar e importar los datos del sistema.
 - **RF8.1:** Exportar e importar en formato XML.
 - **RF8.2:** Exportar e importar en formato JSON.
 - **RF8.3:** Exportar e importar en formato CSV.
- **RF9:** Exportar datos de una cita en específico.
 - **RF9.1:** Exportar en formato HTML.

- **RF9.2:** Exportar en formato PDF.

2.1.4 Sistema de Backups (Copias de Seguridad)

- **RF10:** Gestionar copias de seguridad del sistema.
 - **RF10.1:** Crear una copia de seguridad.
 - **RF10.2:** Restaurar el sistema a partir de una copia de seguridad.
 - **RF10.3:** Borrar una copia de seguridad.

2.1.5 Interfaz y Navegación

- **RF11:** Permitir la navegación por la aplicación mediante un menú estructurado.
- **RF12:** Mostrar un apartado con la información del autor y un enlace directo a su repositorio.

2.1.6 Restricciones de Seguridad

- **RF13:** Restringir que un vehículo pueda tener varias citas en un mismo día.
- **RF14:** Restringir el límite de vehículos a inspeccionar de una misma persona en un mismo día.

2.1.7 Persistencia de Datos

- **RF15:** Almacenamiento de la información compatible con varios motores.
 - **RF15.1:** Operar e intercambiar datos utilizando ADO.NET.
 - **RF15.2:** Operar e intercambiar datos utilizando Dapper.
 - **RF15.3:** Operar e intercambiar datos utilizando Entity Framework Core.

2.1.8 Configuraciones del Sistema

- **RF16:** Modificar la configuración del sistema mediante un archivo:
 - Modificar el tipo de borrado que se debe implementar.
 - Modificar el tipo de motor que se quiera utilizar.
 - Modificar el formato en el que se realizan las copias de seguridad.

2.2 Requisitos No Funcionales (RNF)

2.2.1 Tecnologías y Entorno

- **RNF 1:** El sistema debe estar desarrollado exclusivamente con el lenguaje de programación C# 14.
- **RNF 2:** La aplicación debe ejecutarse sobre la plataforma .NET 10.
- **RNF 3:** El entorno gráfico de escritorio debe ser construido utilizando WPF y XAML.

2.2.2 Buenas Prácticas y Arquitectura

- **RNF 4:** El código fuente debe aplicar estrictamente los principios de diseño SOLID para garantizar su mantenibilidad.
- **RNF 5:** El sistema debe seguir el patrón de Arquitectura Limpia (Clean Architecture), manteniendo las capas de presentación, negocio y datos desacoplados.
- **RNF 6:** El desarrollo debe gestionarse en un repositorio de GitHub utilizando obligatoriamente la metodología GitFlow.

2.2.3 Restricciones Técnicas

- **RNF 7:** El listado de citas en la interfaz gráfica debe optimizarse mediante paginación.
- **RNF 8:** Los componentes de datos deben ser intercambiables.
- **RNF 9:** Implementación de logs en el sistema mediante consola y archivo. Los archivos de log del sistema almacenados en disco deben tener una persistencia máxima de 5 días.
- **RNF 10:** La información exportada debe almacenarse bajo estándares invariables de la industria, utilizando el formato ISO correspondiente.

2.3 Requisitos de Información (RI)

2.3.1 Modelo de Dominio

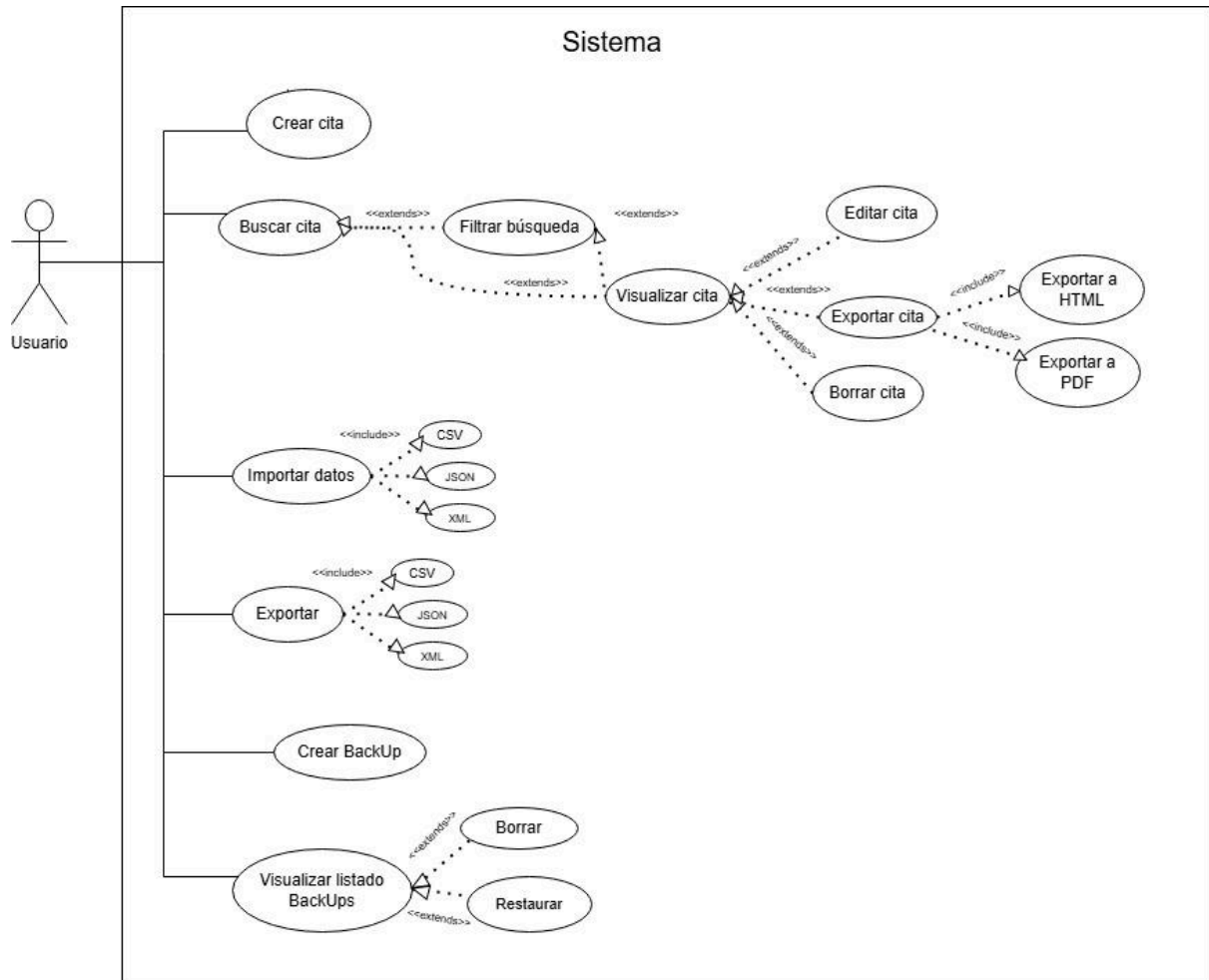
- **RI 1:** Las citas deben contemplar los siguientes datos y restricciones de negocio:
 - **Id:** Clave primaria autonumérica (por una cuestión de agilidad y eficiencia).
 - **Fecha de Inspección:** **DateTime** (*not null*). Si se va a registrar va a haber una cita. Debe estar comprendida entre la fecha actual y un máximo de 30 días posteriores.
 - **Fecha de Matriculación:** **DateTime** (*not null*). Tiene que estar registrado, sino no sería un vehículo legal. No puede ser una fecha futura a la actual.
 - **Matrícula:** **String** de 7 caracteres. Formato: 4 números seguidos de 3 letras mayúsculas. Letras excluidas: I, O, Ñ, Q, U, V, W, X, Y, Z.
 - **Dni del dueño:** **String** de 9 caracteres. Debe cumplir con el algoritmo oficial para detectar si un DNI es verdadero y el formato de 8 números y 1 letra.
 - **Modelo:** **String** de 100 caracteres de longitud máxima.
 - **Marca:** **String** de 100 caracteres de longitud máxima.
 - **Motor:** **Enum** (Gasolina, Diésel, Eléctrico, Hidrógeno).
 - **Cilindrada:** **Double**.
 - **Metadatos de Auditoría (IsDeleted):** **bool**. Necesario para el borrado lógico.
 - **Metadatos de Auditoría (CreatedAt):** **DateTime**. Auditoría.
 - **Metadatos de Auditoría (UpdatedAt):** **DateTime**. Auditoría.
- **Nota:** Se prescinde del campo **DeletedAt** porque el **UpdatedAt** ya reflejaría la hora del borrado lógico en su última actualización, y el borrado físico no lo necesita.

2.3.2 Configuración del Sistema

- **RI2:** El sistema debe almacenar y leer un objeto de información estructurado en un archivo JSON local con los siguientes datos:
 - **Nombres de archivos y directorios:** **Strings**.
 - **Tipo de borrado:** **Bool** (true para borrado lógico y false para borrado físico).
 - **Poblar la base de datos:** **Bool** (indica si el sistema debe ejecutar el seeder para rellenar la base de datos con datos de prueba al arrancar).
 - **Conexión de motores de memoria persistente:** **String** (cadena de conexión compartida o específica para ADO.NET, Dapper y Entity Framework Core).
 - **Configuración del log:** **Objeto/String** que define la ruta del archivo de texto del log y el nivel mínimo de registro.

3. DOCUMENTACIÓN UML

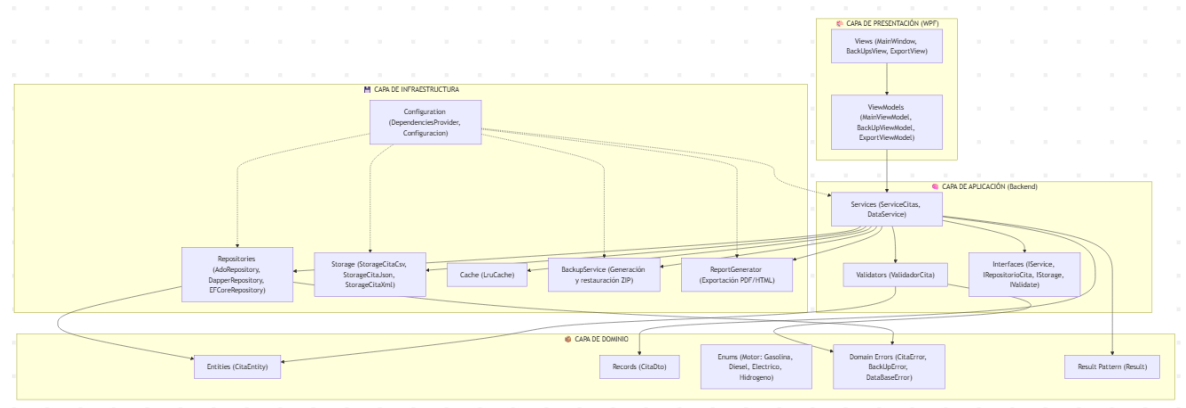
3.1 Diagrama de Casos de Uso



Este diagrama representa las acciones que un usuario puede realizar mediante el programa, contando con funcionalidades de gestión de citas que en su mayoría se sustentan de la función de búsqueda ya que, para poder operar con una cita, es necesario conocer la cita en primer lugar.

Además de las funciones de las citas, cuenta con operaciones de exportación, importación y gestión de copias de seguridad, estando la visualización del listado de BackUps estructurada de una manera similar a la búsqueda y visualización de citas.

3.2 Diagrama de Arquitectura



Este diagrama representa la arquitectura del programa, diseñada por capas, y consta de:

- **Capa de Presentación:** Referente a todo lo que el usuario puede ver y su lógica.
- **Capa de Aplicación:** Orquesta la lógica de negocio.
- **Capa de Dominio:** Representa la información con la que el programa va a trabajar.
- **Capa de Infraestructura:** Implementa los detalles técnicos (acceso a datos, persistencia, configuración).

Se puede observar cómo la interfaz hace uso de la capa de aplicación mediante los servicios, cómo los servicios hacen uso de los modelos de dominio para realizar las operaciones pertinentes, y cómo la infraestructura termina el trabajo con la configuración y el inyector de dependencias proveyendo de las clases y campos necesarios tanto para implementaciones en la infraestructura como para la creación de los servicios.

Esta arquitectura permite:

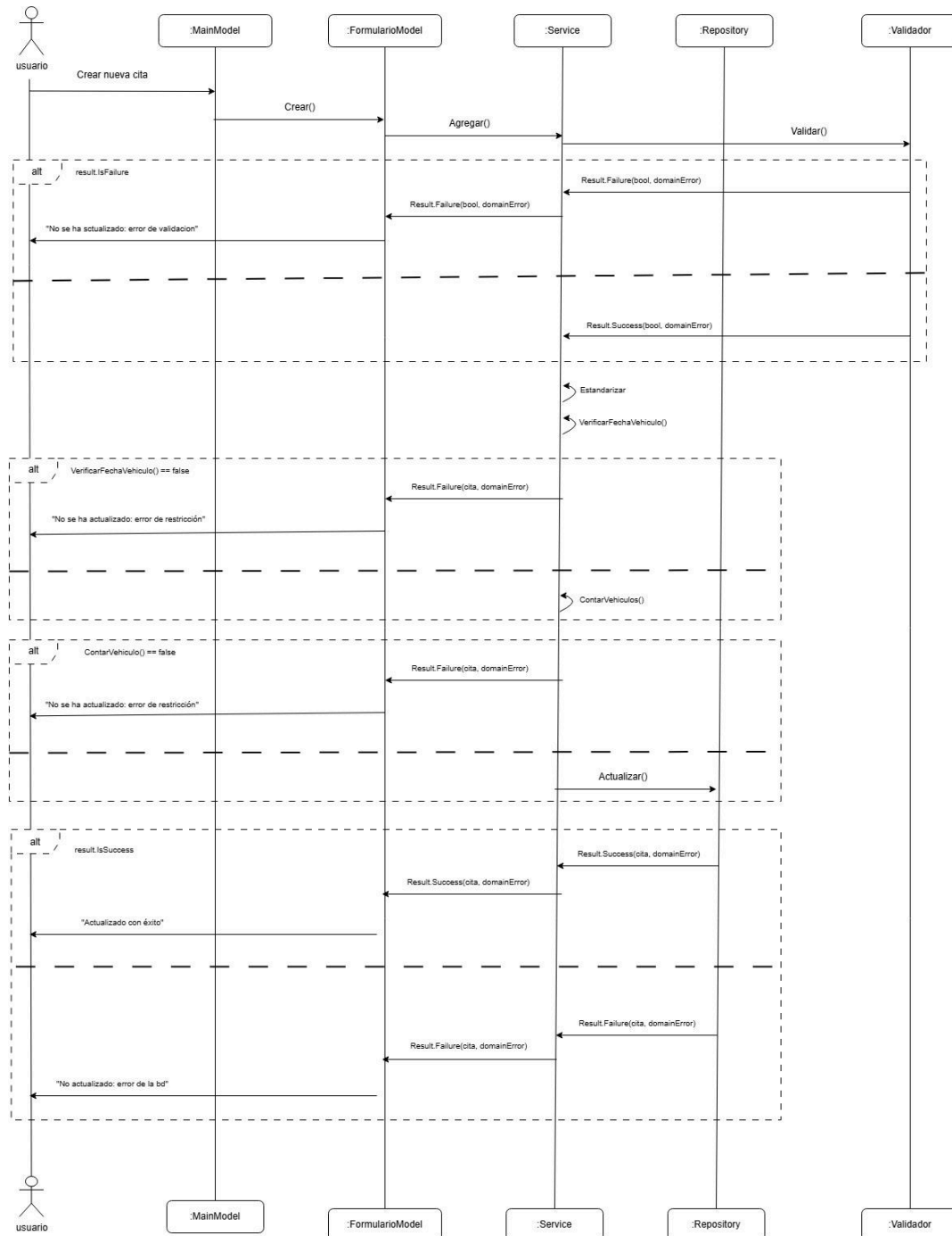
- Separación clara de responsabilidades entre capas.
- Independencia entre la presentación y la lógica de negocio.
- Múltiples implementaciones de acceso a datos sin afectar al resto del código.
- Facilidad para testing mediante inyección de dependencias.
- Escalabilidad y mantenibilidad del código.
- Cambios en la interfaz sin afectar a la lógica de negocio.

Nota sobre el diseño: Se puede observar la implementación de una caché que no aparece en la especificación de requisitos. En un primer momento se había implementado para optimizar el `GetById`, pero durante el desarrollo se vio que este método no es usado. Se ha quitado su implementación del código ya que no aportaba nada, pero se mantiene en el diagrama por si a futuro se pudiese utilizar.

3.3 Diagrama de Secuencias

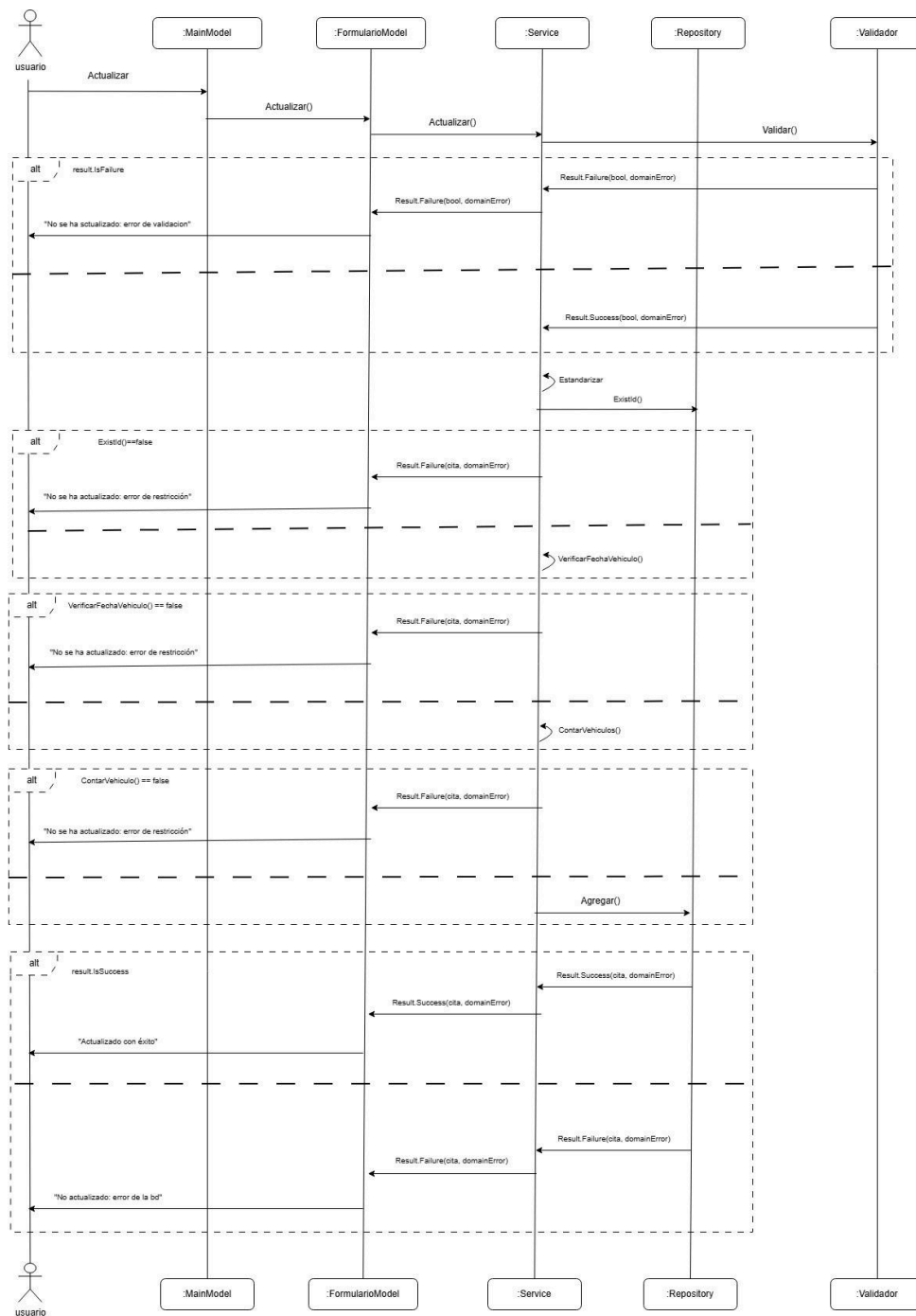
Diagramas de secuencias de las funciones CRUD utilizadas para mostrar el flujo principal del programa:

3.3.1 Diagrama Agregar



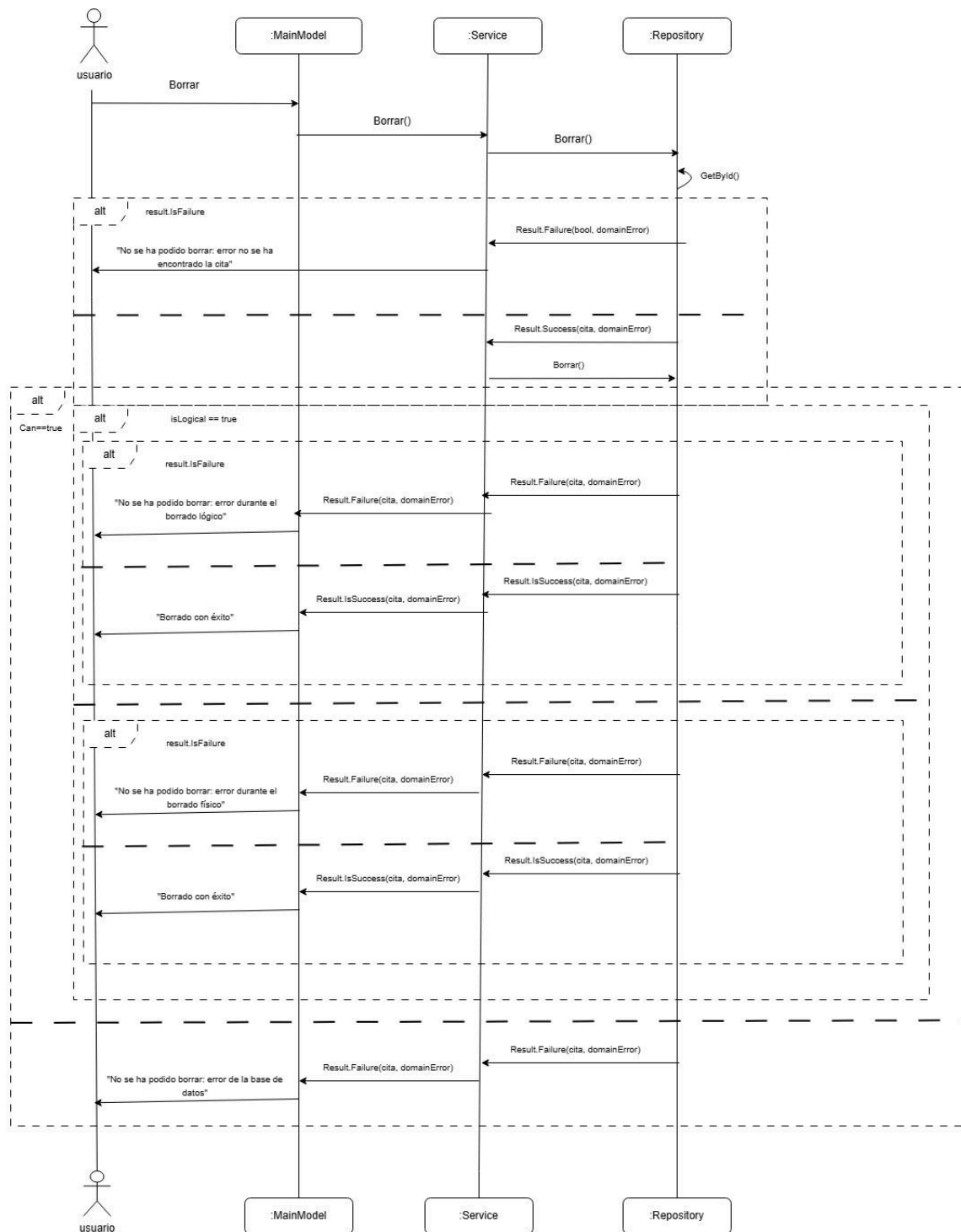
Muestra el flujo de la función de Agregar. Desde la página principal se crea el formulario; una vez completado, se llama al servicio con la cita creada, se valida, se estandariza y se asegura de que la cita no viole las restricciones de integridad antes de pasar al repositorio. Una vez dentro de este, se guarda en la base de datos tras realizar las operaciones necesarias para una persistencia adecuada. En el caso de que algo fallase, devolvería un error de dominio y, si la función tiene éxito, devuelve la cita.

3.3.2 Diagrama Actualizar



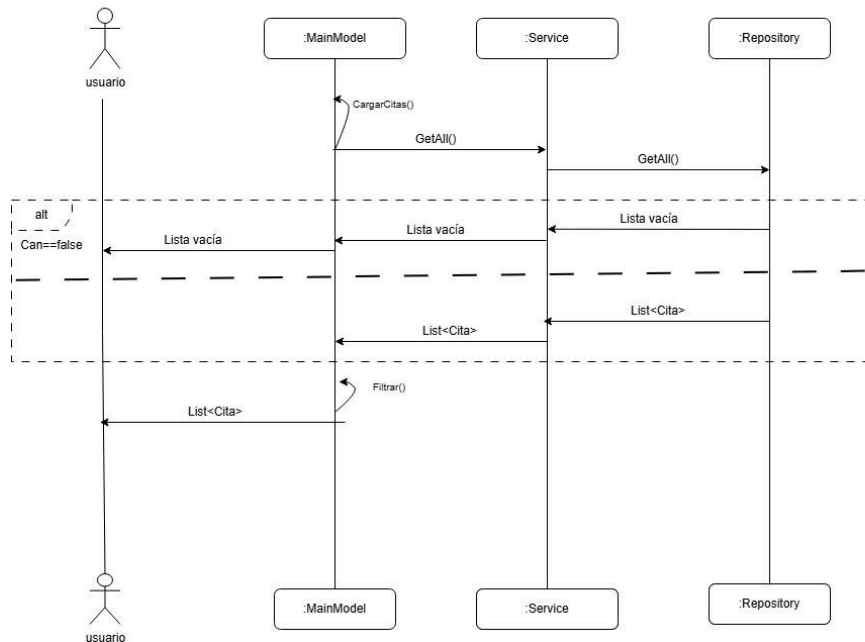
Muestra el flujo de la función de Actualizar. Desde la página principal se crea el formulario; una vez completado, se llama al servicio con la cita creada, se valida, se estandariza y se asegura de que la cita no viole las restricciones de integridad antes de pasar al repositorio. Una vez dentro de este, se guarda en la base de datos tras realizar las operaciones necesarias para una persistencia adecuada. En el caso de que algo fallase, devolvería un error de dominio y, si la función tiene éxito, devuelve la cita.

3.3.3 Diagrama Borrar



Muestra el flujo de la función de Borrar. Desde la página principal se llama a la función de borrado del servicio, que a su vez llama a la función de borrado del repositorio. En el repositorio primero se comprueba que la cita que se quiera borrar exista para evitar problemas de integridad; posteriormente se borra de una manera o de otra dependiendo del flag **isLogical**. Si es un borrado lógico, se cambian los metadatos para que aparezca como tal mediante un update. Si es un borrado físico, se borra permanentemente de la base de datos. En el caso de que algo fallase, devolvería un error de dominio y, si la función tiene éxito, devuelve la cita.

3.3.4 Diagrama GetAll



Muestra el flujo de la función **GetAll**. El usuario no necesita pedir la información en un primer momento: es la clase la que, al crearse, inicializa la función para obtener un listado de citas. La vista principal (ViewModel) llama al servicio y este llama al repositorio. Si en el repositorio algo sale mal, este devuelve una lista vacía y, si no, el listado pedido. Luego, la vista (ViewModel) comienza a filtrar para finalmente darle al usuario el listado deseado definitivo.

A tener en cuenta: Lo lógico al pensar en realizar diagramas de flujos de operaciones CRUD sería asumir los siguientes métodos: Agregar, Borrar, Actualizar y GetById. En vez de este último, he agregado el **GetAll** ya que el **GetById** no se utiliza para buscar nada (el usuario no interactúa con él, sino que sirve como método interno para obtener citas sin repetir bloques de código y es funcional por si un día se requiere). Al no ser usado por el usuario, creí más conveniente intercambiarlo por el **GetAll**, ya que el flujo de la gestión de citas (excepto por añadir) y su visualización están condicionados a esta función.

4. DISEÑO DE LA BASE DE DATOS

4.1 Elección de SQLite

Para la base de datos se ha elegido SQLite por fines didácticos, con el objetivo de aprender a mapear usando entidades al tener las restricciones de datos limitados de este motor. Por otro lado, al ser una aplicación pequeña, la base de datos solo cuenta con una tabla llamada **Cita**.

4.2 Estructura de la Tabla Cita

SQL

- CREATE TABLE IF NOT EXISTS Cita (
- Id INTEGER PRIMARY KEY,
- FechaMatriculacion TEXT NOT NULL,
- FechaInspeccion TEXT NOT NULL,
- Matricula TEXT NOT NULL,
- Modelo TEXT NOT NULL,
- Marca TEXT NOT NULL,
- Motor TEXT NOT NULL,
- Cilindrada REAL CHECK (Cilindrada > 0) NOT NULL,
- DniDueno TEXT NOT NULL,
- IsDeleted INTEGER DEFAULT 0,
- CreatedAt TEXT NOT NULL,
- UpdatedAt TEXT NOT NULL,
- UNIQUE(Matricula, FechaInspeccion)
-);

4.3 Restricciones y Validaciones

La tabla **Cita** representa el modelo de dominio cita. Cuenta como clave primaria con un Id autonumérico para una mayor comodidad, aunque se podría haber usado la combinación de matrícula y fecha de inspección como clave primaria.

Las restricciones estructurales en esta tabla sirven para asegurarse de que los datos de la cita no lleguen nulos, ya que ninguno de ellos debería de serlo. Aquellos datos que no dependen de la interfaz para conseguir su valor no han sido restringidos de tal manera en la base de datos, ya que no es necesario. Además, se cuenta con una restricción adicional en el campo **Cilindrada** para que cumpla el requisito lógico de que no sea menor o igual que cero. El resto de las restricciones son implementadas en diferentes capas de la arquitectura para obtener una mayor cobertura y seguridad.

4.4 Clave Alternativa (Matrícula, FechaInspeccion)

La combinación de matrícula y fecha de inspección se utiliza conceptualmente en varias partes del programa (principalmente en logs y métodos de búsqueda) como referencia alternativa para mejorar la legibilidad y así hacer el programa más mantenible. Esta es una idea de mejora futura para asegurar su persistencia estricta.

4.5 Consideraciones Futuras

- **Nota de refactorización:** El campo **DniDueño** ha causado algunos problemas por el uso de la letra 'ñ'. Se tendrá en cuenta para versiones futuras como candidato a ser refactorizado.

5. TESTING

5.1 Metodología de Testing

Los métodos de testing utilizados en este proyecto son **pruebas unitarias** y **test con dobles** para clases con dependencias. Al disponer del código fuente, se han realizado **pruebas de caja blanca** aplicando la fórmula de complejidad ciclomática simplificada.

5.2 Complejidad Ciclomática

La fórmula aplicada es:

$$V(G) = P + 1$$

Donde P es el número de nodos predicado y V(G) es la complejidad resultante.

A continuación se expone el ejemplo de cómo se ha establecido el número de tests mínimos, así como el tipo de test a realizar usando un grafo y la fórmula para la **Función Agregar() del servicio**: En este caso, el grafo contaría con 4 nodos predicados y, por lo tanto, siguiendo la fórmula, se tienen que hacer un mínimo de 5 tests.

5.3 Casos de Prueba (Función Agregar)

NºTest	Escenario	Entrada	Resultado Esperado
Test 1	Agregar cita válida	Cita con matrícula válida, DNI correcto, fecha disponible y propietario con menos de 3 citas el mismo día.	La cita se valida, se estandariza y se guarda correctamente en el repositorio.
Test 2	Error de validación	Cita con datos inválidos (ejemplo: matrícula incorrecta o DNI inválido).	El validador devuelve un error de dominio y la cita no se persiste.
Test 3	Fecha ya ocupada para el vehículo	Cita cuya matrícula ya tiene una inspección asignada en la misma fecha.	Se devuelve el error FechaYaEstablecida y la operación se cancela.

Test 4	Límite de vehículos por propietario excedido	Propietario con 3 o más citas registradas para el mismo día.	Se devuelve el error OwnerWithThreeOrMoreCitas y la cita no se guarda.
Test 5	Error en persistencia	Cita válida pero el repositorio falla durante el guardado.	El sistema devuelve el error correspondiente de persistencia y registra el fallo en el log.

5.4 Cobertura de Código

Cabe aclarar que se han excluido del análisis de cobertura:

- Views (código WPF generado, difícil de testear).
- Providers y Factories (inyección de dependencias).
- Código generado automáticamente.

En las clases testables se alcanza una cobertura del **81% de líneas** y **75% de ramas**, con especial énfasis en la lógica crítica de negocio.

Cobertura Desglosada por Capas:

- **Servicios:** 81.3%
- **Validadores:** 97.6%
- **Mappers:** 100.0%
- **Models:** 98.3%
- **Repositories:** 75.0% - 77.0%
- **Storage:** 80.0% - 82.0%
- **Cache:** 82.8%

Nota: Se pueden revisar estos porcentajes en la carpeta de **coveragereport** en el proyecto **ITV.Test** visualizándolos mediante el fichero **index.html**.

6. ANÁLISIS ECONÓMICO

A continuación se detalla el presupuesto y la estimación de esfuerzos para las fases de desarrollo del sistema:

Componente / Fase de Desarrollo	Horas Estimadas	Precio / Hora	Total
Análisis del Sistema	20h	20,00 €	400,00 €
Documentación Técnica (UML)	15h	20,00 €	300,00 €
Desarrollo Backend & Reglas de Negocio	65h	20,00 €	1.300,00 €
Control de Calidad (Testing & QA)	25h	20,00 €	500,00 €
Interfaz Gráfica de Escritorio (WPF/XAML)	15h	20,00 €	300,00 €
Configuración de Infraestructura & Motores	10h	20,00 €	200,00 €
PRESUPUESTO TOTAL ESTIMADO	150h	20,00 €	3.000,00 €

7. MEJORAS A FUTURO

Para finalizar, aparte de las ideas de mejora ya mencionadas a lo largo del documento, se plantean las siguientes optimizaciones y refactorizaciones:

7.1 Optimizaciones Arquitectónicas

- **Migración de Restricciones al Repositorio:** Cambiar las restricciones para que estén en el repositorio una vez la app escale y llevar todos los datos del repositorio al servicio sea más costoso.
- **Paginación en el Repositorio:** Cambiar la paginación para que también se incluya en el repositorio por la misma razón de escalabilidad.
- **Paginación en DataService:** En las operaciones del **DataService** que implican obtener todos los datos del servicio, realizar el proceso también aplicando paginación.
- **Caché LRU en GetAll:** Implementar la caché LRU en el **GetAll** para darle un uso práctico; como idea, en vez de guardar una sola cita, podría almacenar la colección de una paginación por si se quisiese volver a ella de una forma más eficiente.

7.2 Testing y Código

- **Refactorización de Tests:** Limpiar los tests unitarios debido a que actualmente hay bastante código duplicado que se podría evitar.

7.3 Base de Datos

- **Restricciones Adicionales:** Añadir más restricciones directamente en el motor de la base de datos para robustecer la integridad.

7.4 Interfaz de Usuario

- **Mejora de la Arquitectura del Frontend:** Mejorar el diseño del frontend de la aplicación añadiendo reactividad.
- **Filtro de Citas Borradas:** Permitir que el usuario elija de forma explícita en la interfaz si se quieren mostrar o no las citas borradas mediante borrado lógico.

7.5 Datos y Validación

- **Seeder Mejorado:** Optimizar la función del seeder encargado de rellenar los datos de prueba, añadiendo más flags y condiciones de inicialización.
- **Validador Refactorizado:** Refactorizar el componente validador para que devuelva una lista de cadenas de texto (*strings*) en vez de uno solo, permitiendo ofrecer un informe más detallado al usuario sobre todos los datos que está insertando de forma errónea simultáneamente.

8.Links

Video: [DemoFuncional ITV](#)

Repositorio: [LSPlaura/ITV](#) y [LSPlaura/BackITV](#) este último es el primero con el que estuve trabajando hasta que me di cuenta que no me dejaría hacer bien los test por la forma en la que a github la estructura de carpetas y tuve que crear otro repositorio.